



รายงาน

เรื่อง To-Do List Application

จัดทำโดย

นางสาวชลิตา วายุโชติ 660910486

นายทิวากร กรดเครือ 660910742

นางสาวรวิณัฏฐา หล้าแพร่ 660910745

อาจารย์ที่ปรึกษา

อาจารย์ ดร.ศักดิ์ระพี ไพศาลนันท์

รายวิชา 618498-165 เรื่องคัดเฉพาะทางวิศวกรรมอิเล็กทรอนิกส์และคอมพิวเตอร์ 1

ภาคการศึกษาที่ 1 ปีการศึกษา 2569

สาขา อิเล็กทรอนิกส์และระบบคอมพิวเตอร์

ภาควิชาวิศวกรรมไฟฟ้า คณะวิศวกรรมศาสตร์และเทคโนโลยี

มหาวิทยาลัยศิลปากร วิทยาเขตสนามจันทร์

1. บทนำและวัตถุประสงค์ (Introduction & Objectives)

โปรเจกต์นี้เป็นการพัฒนาเว็บแอปพลิเคชันจัดการรายการสิ่งที่ต้องทำ (To-Do List) แบบ Full-Stack โดยมีการเชื่อมต่อระหว่างส่วนหน้าบ้าน (Frontend) และหลังบ้าน (Backend) ผ่าน RESTful API เพื่อจัดการข้อมูลรายการภารกิจแบบเรียลไทม์

วัตถุประสงค์

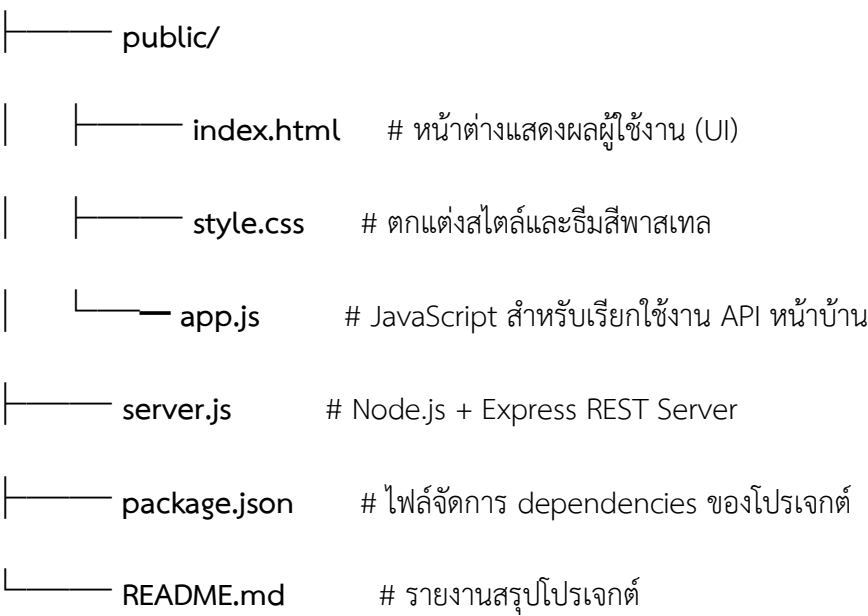
1. เพื่อศึกษากระบวนการพัฒนา Web Application ทั้งส่วน Frontend และ Backend
2. เพื่อเรียนรู้การสร้าง RESTful API ด้วย Node.js และ Express.js
3. เพื่อฝึกฝนการจัดการไฟล์ static (HTML/CSS/JS) และการเชื่อมต่อ API ด้วย Fetch API

2. เครื่องมือเทคโนโลยีที่ใช้ (Tech Stack)

1. **Frontend:** HTML5, CSS3 (การตกแต่งสไตล์โทนสีชมพู-ม่วงพาสเทล), JavaScript (ES6)
2. **Backend:** Node.js, Express.js
3. **Middleware & Libraries:** cors, body-parser, nodemon (สำหรับ Development)
4. **Version Control:** Git & GitHub Repository

3. โครงสร้างโปรเจกต์ (Project Structure)

todo-app/



4. รายละเอียดการทำงานของ RESTful API (API Endpoints)

เซิร์ฟเวอร์ทำงานที่ <http://localhost:3000> โดยมี API ในการจัดการรายการงานดังนี้

Method	Endpoint	รายละเอียดการทำงาน	Status Code สำเร็จ
GET	/api/tasks	ดึงรายการสิ่งที่ต้องทำทั้งหมด	200 OK
POST	/api/tasks	เพิ่มรายการสิ่งที่จะทำใหม่	201 Created
PATCH	/api/tasks/:id	แก้ไขข้อมูลรายการ (เปลี่ยนสถานะ/ชื่อ)	200 OK
DELETE	/api/tasks/:id	ลบรายการตาม ID ที่ระบุ	204 No Content

5. ตัวอย่างโค้ดระบบหลังบ้าน (server.js)

```
const express = require('express');

const cors = require('cors');

const bodyParser = require('body-parser');

const app = express();

const PORT = 3000;

app.use(cors());

app.use(bodyParser.json());

app.use(express.static('public'));

let tasks = [

  { id: 1, title: 'ส่งรายงานโปรเจกต์', category: 'เรียน', done: false }

];

// 1. GET /api/tasks

app.get('/api/tasks', (req, res) => {

  res.status(200).json(tasks);

});

// 2. POST /api/tasks

app.post('/api/tasks', (req, res) => {

  const { title, category } = req.body;

  const newTask = {

    id: Date.now(),

    title: title || 'ไม่มีชื่อรายการ',

    category: category || 'ทั่วไป',

    done: false

  };

  tasks.push(newTask);

  res.status(201).json(newTask);
```

```
});
```

```
// 3. PATCH /api/tasks/:id
```

```
app.patch('/api/tasks/:id', (req, res) => {
```

```
  const id = parseInt(req.params.id);
```

```
  const task = tasks.find(t => t.id === id);
```

```
  if (!task) return res.status(404).json({ message: 'ไม่พบรายการที่ระบุ' });
```

```
  if (req.body.title !== undefined) task.title = req.body.title;
```

```
  if (req.body.category !== undefined) task.category = req.body.category;
```

```
  if (req.body.done !== undefined) task.done = req.body.done;
```

```
  res.status(200).json(task);
```

```
});
```

```
// 4. DELETE /api/tasks/:id
```

```
app.delete('/api/tasks/:id', (req, res) => {
```

```
  const id = parseInt(req.params.id);
```

```
  const index = tasks.findIndex(t => t.id === id);
```

```
  if (index === -1) {
```

```
    return res.status(404).json({ message: 'ไม่พบรายการที่ระบุ' });
```

```
  }
```

```
  tasks.splice(index, 1);
```

```
  res.status(204).send();
```

```
});
```

```
app.listen(PORT, () => {
```

```
  console.log(`Server is running on http://localhost:${PORT}`);
```

```
});
```

6. ขั้นตอนการติดตั้งและรันใช้งาน (Setup & Installation)

1. ติดตั้ง dependencies ที่จำเป็น:

```
npm install express cors body-parser
```

```
npm install --save-dev nodemon
```

2. สั่งเปิดใช้งาน Server:

```
node server.js
```

3. เข้าใช้งานผ่านเว็บเบราว์เซอร์:

พิมพ์ URL <http://localhost:3000>

7. สรุปผลการทดสอบระบบ (Conclusion & Results)

1. ระบบสามารถรัน Web Server บน พอร์ต 3000 ได้อย่างถูกต้อง
2. การรับ-ส่งข้อมูลผ่าน REST API ในส่วนของ GET, POST, PATCH, และ DELETE ทำงานร่วมกับหน้าเว็บ Frontend ได้อย่างสมบูรณ์
3. โค้ดทั้งหมดได้รับการบันทึกและจัดการเวอร์ชันอยู่บน GitHub Repository เป็นที่เรียบร้อย

